# General Key Recovery Attack on Pointwise-Keyed Functions

## Application to Alternating Moduli Weak PRFs*

Antoine Sidem[0009−0001−2540−1244] and Qingju Wang[0000−0003−4565−8394]

Télécom Paris, Institut Polytechnique de Paris, Palaiseau, France
antoine.sidem@telecom-paris.fr, qingju.wang@telecom-paris.fr

**Abstract.** The increasing use of multi-party computation (MPC) has spurred the design of symmetric key primitives specifically suited for MPC environments. Recently, weak pseudorandom functions (wPRFs) based on the alternating moduli paradigm have been proposed as a promising class of MPC-friendly primitives. The wPRF proposed at CRYPTO 2024, in its One-to-One parameter set, has been shown to be vulnerable to a key recovery attack dubbed Zeroed-Out, exploiting collisions in the queries.
In this paper, we identify that the aforementioned wPRFs conform to a specific structure, called *pointwise-keyed function*, and further show a different, general key recovery attack. This method, applied to wPRFs in the One-to-One parameter set attacked by Zeroed-out, improves upon the complexity and achieves an attack with complexity below the birthday bound, and stays effective against the proposed countermeasures. For the first time, it succeeds in attacking one of the two Many-to-One parameter sets and stays effective against one of the proposed countermeasures. We also consider its applicability to the alternative wPRF of similar structure proposed by Boneh *et al.* at TCC 2018.

**Keywords:** Weak pseudorandom functions · Alternating moduli · Pointwise multiplication · Key recovery

## 1 Introduction

Pseudorandom functions (PRFs) [GGM86] are fundamental primitives in symmetric cryptography. In particular, they yield direct solutions to most central goals of symmetric cryptography, such as encryption and authentication [BCK96,Bel06]. PRFs have also been widely used as building blocks to construct more advanced cryptographic primitives such as digital signature and indistinguishability obfuscation [Gol86,App14]. They are well studied in the theoretical community and widely used in practice.

As devices constrained to lightweight computation and more advanced real-world applications such as multi-party computation (MPC) become popular,

* This paper was accepted to ASIACRYPT 2025, and will appear in the proceedings.

the efficiency of cryptographic implementations also become increasingly significant. To obtain a better trade-off between security and efficiency, a weaker notion of standard/strong PRFs called weak pseudorandom functions (wPRF) was required. Many solutions have been proposed to address this new need in various applications, such as the leakage-resilient PRFs proposed by Dodis and Pietrzak [DP10], the wPRFs providing low circuit depth proposed by Akavia *et al.* [ABG+14], and more recent proposals that are friendly to MPC protocols following the alternating moduli paradigm proposed by Boneh *et al.* [BIP+18]. This paradigm alternates simple linear operations over different moduli, building low depth-2 wPRF that can be evaluated in a single round communication after preprocessing, which is efficient in MPC settings. A loosely similar paradigm called Learning with Physical Rounding (LWPR) has also developed wPRF proposals in [DMMS21,HMM+23]. Dinur *et al.* [DGH+21] further extended the alternating moduli paradigm and proposed more primitives like one-way functions (OWFs), pseudorandom generators (PRGs) and wPRFs. PRFs of the alternating-moduli paradigm have been used in the oblivious setting ([BCM+24,CDD+24]) and with FHE [ADDG24]. However, protocols built upon these primitives often underperform compared to state-of-the-art. At CRYPTO 2024, Alamati *et al.* [APRR24] revisited the alternating moduli paradigm to propose a new wPRF that improves on previous constructions in terms of efficiency and practicality.

wPRFs are defined as functions that maintain pseudorandomness only when evaluated on uniformly random inputs. While wPRFs are particularly useful in certain cryptographic protocols, their security guarantees are weaker than those of the standard PRFs, making them potential targets for attacks. The security analysis of wPRFs is particularly relevant in settings where the adversaries have partial knowledge over the distribution of the inputs, such as distinguishing attacks, implementations, and side-channel attacks in practical settings. For instance, distinguishing attacks on the wPRFs in [BIP+18] were presented by Cheon *et al.* [CCKK21] with lower complexity than claimed, and they further provided countermeasures to protect against their attacks. Later in [JMN23], it was argued that the fixes suggested by Cheon *et al.* cannot provide the security as claimed.

Very recently, key recovery attacks on the latest wPRFs proposed by Alamati *et al.* [APRR24] are given by Manterola Ayala and Raddum [MR25], which is called Zeroed-out attack in the rest of the paper. They exploit collisions caused by the key-input operation $k \odot_p x$ to gradually distinguish bits that are zero in the key. The Zeroed-out attack on the "One-to-One" Standard alternating moduli $(\mathbb{F}_2, \mathbb{F}_3)$-wPRF leverages collisions to reveal unset bits in the key. In doing so, they query the wPRF for a large number of random inputs, collect them in a table, and check for collisions within. They recover the full key in $\mathcal{O}(2^{0.5\lambda} \log_2 \lambda)$. For the "One-to-One" Reversed alternating moduli $(\mathbb{F}_3, \mathbb{F}_2)$-wPRF, they recover the full key in time $\mathcal{O}(2^{0.84\lambda} \log_2 \lambda)$.

*Motivation.* The main limitation of Zeroed-out attack on the wPRFs is that the complexity is slightly over the birthday bound. Moreover, Zeroed-out is ineffec-

tive against both the standard and the reversed variants within the "Many-to-One" parameter category.

We identify that the wPRFs proposed in [APRR24] and the alternative wPRFs proposed in [BIP+18] conform to a specific structure, which we define as follows:

**Definition 1 (Pointwise-keyed function).** *Let $\mathbb{F}_p, \mathbb{F}_q$ finite fields of respective sizes $p, q$. A pointwise-keyed function $F$ is such that*

$$\begin{aligned} F : \mathbb{F}_p^n \times \mathbb{F}_p^n &\to \mathbb{F}_q^t \\ k, x &\mapsto f\left(k \odot_p x\right) \end{aligned}$$

*where $f : \mathbb{F}_p^n \to \mathbb{F}_q^t$, and $\odot_p$ denotes the pointwise (or Hadamard) multiplication between vectors in $\mathbb{F}_p$.*

The wPRFs proposed in [APRR24] and the alternative wPRFs proposed in [BIP+18] specify the above general function $f$. We show that the pointwise multiplication between vectors over $\mathbb{F}_p$ utilized in the wPRFs is vulnerable to a new general key recovery attack not exploiting collisions in the queries as in Zeroed-out attack.

*Contribution.* In this paper, we present a general key recovery attack on this class of weak PRFs:

- It improves upon the time, data, and memory complexities for the wPRFs attacked in Zeroed-out, ultimately achieving an attack below the birthday bound: the complexity for "One-to-One" Standard and Reversed variants is $\mathcal{O}(2^{0.45\lambda})$ and $\mathcal{O}(2^{0.56\lambda})$ respectively.
- It relies on a different attack angle, querying for low-weight inputs instead of collisions. Thus, it is sensitive to the wPRF's input field and size, rather than its output.
- It successfully attacks a parameter set, i.e., the Standard "Many-to-One", against which Zeroed-out attack was ineffective.
- It stays successful against one of the countermeasures for Zeroed-out.
- Like Zeroed-out attack, it can generalize to any pointwise-keyed wPRF, no matter the internal $f$ function.

Our implementation code can be accessed at:

https://gitlab.telecom-paris.fr/c2/pointwise-keyed-wprf-attack

Moreover, we discuss the potential ways to restore the security of the wPRFs against the general key recovery attacks.

*Outline of the Paper.* The rest of the paper is structured as follows: In Section 2, we recall the definition and security of wPRFs, and we define the Standard and the Reversed wPRF variants in the "One-to-One" and "Many-to-One" category, as proposed in [APRR24]. Section 3 presents our general key recovery attack

against the wPRFs following the form of pointwise-keyed function. In Section 4, we apply the attack to break three out of four wPRF variants in [APRR24], and one of the countermeasures suggested by Zeroed-out paper, and discuss the applicability to the alternative wPRF in [BIP+18]. In Section 5, we compare our attack with Zeroed-out, and we discuss possible ways to restore the security for the wPRFs attacked. Finally, we conclude the paper in Section 6.

## 2 Preliminaries

In this section, we first give the definition and security notions of weak pseudorandom function (wPRF), followed by a brief description of the recommended variants propsoed by Alamati *et al.* [APRR24]. Then we briefly describe the Zeroed-out attack.

### 2.1 Weak Pseudorandom Functions

We use $[n]$ to denote the set $\{1, 2, \ldots, n\}$. For a set $S$, we write $s \xleftarrow{\$} S$ to indicate that $s$ is sampled uniformly at random from $S$. For a random variable $\mathcal{D}$, we write $d \leftarrow \mathcal{D}$ to indicate that $d$ is sampled according to $\mathcal{D}$. A function $f : \mathbb{N} \to [0, 1]$ is said to be negligible if for every $c \in \mathbb{N}$, there exists $N \in \mathbb{N}$ such that for all $n > N$, $f(n) < n^{-c}$, and we write $\mathsf{negl}(\cdot)$ to denote such a function. Let $\lambda$ be *the security parameter*, throughout of the paper. For two ensembles of random variables $\mathcal{D}_0 = \{\mathcal{D}_{0,\lambda}\}_{\lambda\in\mathbb{N}}$, $\mathcal{D}_1 = \{\mathcal{D}_{1,\lambda}\}_{\lambda\in\mathbb{N}}$, we write $\mathcal{D}_0 \approx_c \mathcal{D}_1$ to indicate that for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, it holds that

$$\left| \Pr_{d\leftarrow\mathcal{D}_0}[\mathcal{A}(d) = 1] - \Pr_{d\leftarrow\mathcal{D}_1}[\mathcal{A}(d) = 1] \right| \leq \frac{1}{2} + \mathsf{negl}(\lambda).$$

**Weak Pseudorandom Function (wPRF).** A function $f : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ with the key space $\mathcal{K}$, input space $\mathcal{X}$ and output space $\mathcal{Y}$ is a $(q, t, c)$-wPRF if for running in time bounded by a polynomial $t(\lambda)$ it holds that

$$\{(x_i, f(k, x_i)\}_{i\in[q]} \approx_c \{(x_i, y_i)\}_{i\in[q]},$$

where $k \xleftarrow{\$} \mathcal{K}$, $x_i \xleftarrow{\$} \mathcal{X}$ and $y_i \xleftarrow{\$} \mathcal{Y}$.

At CRYPTO 2024, Alamati *et al.* [APRR24] introduced a novel wPRF tailored for efficient MPC applications. This construction builds upon and generalizes the alternating moduli paradigm initially proposed by Boneh *et al.* [BIP+18].

*Standard wPRF.* The Standard $(\mathbb{F}_2, \mathbb{F}_3)$-wPRF is defined as

$$F(k, x) := \mathbf{B} \cdot_3 \left[\mathbf{A} \cdot_2 (k \odot_2 x)\right],$$

where $\cdot_p$ denotes matrix multiplication modulo $p$, and $\odot_p$ denotes component-wise multiplication modulo $p$. $x, k$ are random binary vectors of length $n$ representing the input and key, $\mathbf{A} \in \mathbb{F}_2^{m\times n}$ is a random matrix, and $\mathbf{B} \in \mathbb{F}_3^{t\times m}$ is a random compressing matrix (i.e., $t < m$).

We denote by $h_i$ the number of elements in $k$ with the value $i$, $i \in \{0, 1\}$. For a key $k$ of length $n = 2\lambda$ which is chosen uniformly at random, it is expected that $h_1 \approx h_0 \approx \lambda$.

*Reversed wPRF.* For scenarios requiring binary secret-sharing outputs, the Reversed Moduli $(\mathbb{F}_3, \mathbb{F}_2)$-wPRF is proposed as

$$F(k, x) := \mathbf{B} \cdot_2 \ [\mathbf{A} \cdot_3 \ (k \odot_3 x)],$$

where $x, k \in \mathbb{F}_3^n$, $\mathbf{A} \in \mathbb{F}_3^{m \times n}$ is a random matrix, and $\mathbf{B} \in \mathbb{F}_2^{t \times m}$ is a random compressing matrix (i.e., $t < m$). Extending the notations for the Standard wPRF, let $h_i$ denote the number of elements in $k$ with the value $i$, $i \in \{0, 1, 2\}$. For a uniformly random key of length $n = 2\lambda$, we expect that $h_0 \approx h_1 \approx h_2 \approx \frac{2\lambda}{3 \log_2 3}$.

The generalized $(\mathbb{F}_p, \mathbb{F}_q)$-wPRF extends this concept to arbitrary primes $p$ and $q$ in the natural way as

$$F(k, x) := \mathbf{B} \cdot_q \ [\mathbf{A} \cdot_p \ (k \odot_p x)],$$

where $x, k \in \mathbb{F}_p^n$, $\mathbf{A} \in \mathbb{F}_p^{m \times n}$ and $\mathbf{B} \in \mathbb{F}_q^{t \times m}$ are random matrices. For the Standard and the Reversed variants, $(p, q) = (2, 3)$ and $(3, 2)$ respectively.

**Parameter Sets.** Alamati *et al.* [APRR24] divide the parameter sets into two groups, namely One-to-One parameters and Many-to-One parameters. We summarize the recommended parameter sets for all wPRFs in Table 1. The authors claim $\lambda$-bit security for each parameter set.

*One-to-One Parameters.* The One-to-One parameter set is designed to give a (roughly) one-to-one mapping between inputs and outputs. Specifically, the input space and output space are of the same size, and for any given input $x$ it was claimed that we can expect a unique output $y$.

*Many-to-One Parameters.* The Many-to-One parameter set has a larger input space than the output space. For any given output $y$ there should be multiple values for $x$ mapping to $y$, leading to a many-to-one mapping between inputs and outputs.

### 2.2 The Zeroed-out Attack

The idea of Zeroed-out is to query the wPRF on random inputs, building up a large table of input and output values $(x, y)$. Zeroed-out analyzed the two variants in the Many-to-One category.

For the Standard variant, each collision in the output $y$ reveals, on average, half of the remaining unset bits. After collecting some number of collisions, the number of revealed zero bits converges, and it becomes likely that nearly all of them have been revealed. Using the birthday paradox, the expected number of samples required to find $c$ collisions is $N = \sqrt{2 \cdot |\operatorname{Im} F| \cdot c}$. In the Standard

Table 1: Recommended parameter sets for the wPRF in [APRR24]. $\lambda$-bit security is claimed for each parameter set.

| Variants | One-to-One | | | Many-to-One | | |
|---|---|---|---|---|---|---|
| | $n$ | $m$ | $t$ | $n$ | $m$ | $t$ |
| Standard $(\mathbb{F}_2, \mathbb{F}_3)$-wPRF | $2\lambda$ | $7.06\lambda$ | $\frac{2\lambda}{\log_2 3}$ | $4\lambda$ | $2\lambda$ | $\frac{\lambda}{\log_2 3}$ |
| Reversed $(\mathbb{F}_3, \mathbb{F}_2)$-wPRF | $\frac{2\lambda}{\log_2 3}$ | $\frac{7.06\lambda}{\log_2 3}$ | $2\lambda$ | $\frac{4\lambda}{\log_2 3}$ | $2\lambda$ | $\lambda$ |

variant, a key $k$ chosen uniformly at random, so we expect $h_0 \approx h_1 \approx \lambda$, then $|\operatorname{Im} F| = 2^{h_1} \approx 2^{\lambda}$, and therefore $N = \sqrt{2^{(\lambda+1)} \cdot c}$. The attack concludes with a short exhaustive search for the remaining bits of the key, when $c = \log_2 \lambda$ the exhaustive search becomes cheaper than finding additional collisions. Consequently, the complexity of the attack is of order $\mathcal{O}(2^{0.5\lambda} \log_2 \lambda)$.

As for the Reversed variant, each collision in the output $y$ is expected to recover approximately 2/3 of the remaining zero bits of the key. The key is uniformly random selected, so we expect $h_0 \approx h_1 \approx h_2 \approx \frac{2\lambda}{3 \log_2 3}$. Similarly to the Standard variant, $|\operatorname{Im} F| = 2^{\log_2(3) \cdot (h_1 + h_2)} \approx 2^{4\lambda/3}$, then the expected number of samples $N = \sqrt{2^{(4\lambda/3+1)} \cdot c}$. Assume after collecting $c$ collisions, we identify all zero bits, i.e., the number of remaining zero bits is less than 1. The minimum expected number of collisions can be solved from the expression $\left(1 - \sum_{i=1}^{c} \frac{2}{3^i}\right) \cdot h_0 \approx \left(1 - \sum_{i=1}^{c} \frac{2}{3^i}\right) \cdot \frac{2\lambda}{3 \log_2 3} < 1$, and it is of order $\mathcal{O}(\log_3 \lambda)$. The total complexity of the collision search thus is $\mathcal{O}(2^{2\lambda/3 \log_3 \lambda})$. After all zero bits are identified, the total number of the remaining key bits need to be exhaustively searched is $2^{n \cdot (2/3)} \approx 2^{(2\lambda/\log_3) \cdot (2/3)} \approx 2^{0.84\lambda}$, which domains the total cost. Therefore the total complexity of this attack is $\mathcal{O}(2^{0.84\lambda})$.

## 3 Partial-Key Search Attack on General Pointwise-Keyed Functions

In this section, we present a general key recovery attack against the wPRFs following the form of pointwise-keyed functions defined in Definition 1, of the form

$$F(x, k) := f\left(k \odot_p x\right),$$

without considering the specification of the function $f$.

### 3.1 Attack Overview

We recall the attack model against a weak-PRF $F$: we, the attacker, must recover the secret key, given query access to an oracle dispensing input-output pairs $x, y = F(k, x)$ where $x$ is *uniformly random*. Thus we do not have any control or adaptability margin in the querying process.

If we obtain a wPRF input-output pair $(x, y)$ with low Hamming weight $\mathsf{hw}(x) = s$, then we know that only $s$ elements of the key intervene in the function computation. We can perform exhaustive search on the partial-key corresponding to those $s$ elements, and find at least one solution for the partial-key. This is done by testing $y \stackrel{?}{=} F(\hat{k}, x)$ for each candidate $\hat{k}$, that contains the $s$ tested elements at positions where $x_i \neq 0$. The value of $\hat{k}_i$ where $x_i = 0$ is unimportant, as they are not involved in the function computation; by convention we set them to zero.

If a single candidate partial-key $\hat{k}$ passes the exhaustive search, we can be sure it is correct, and we learn $s$ key elements. We repeat this attack a finite number of times to eventually learn the entire key. We call each of such partial key recoveries a *round*.

**Property 1** (Online attack)**.** The attack as described here is purely online and requires constant memory, if we ignore occurrences where there are multiple $\hat{k}$ that pass the filter $y \stackrel{?}{=} F(\hat{k}, x)$. However, it is also possible to save, then leverage, such ambiguous candidate lists. The number of candidates depends on the relative sizes of the function's input and image spaces. For wPRFs with similarly sized inputs and outputs, the attack can be effectively memory-less. We discuss the number of passing candidates, and how to leverage them, in Section 3.6.2.

**Property 2** (Lower weight input is better)**.** If somehow we obtain a pair $(x, y)$ such that $\mathsf{hw}(x) = s' < s$, we can search for candidate partial-keys with cost $p^{s'}$ and possibly learn $s'$ key elements. We have $\frac{s'}{p^{s'}} > \frac{s}{p^s}$, thus the cost-benefit ratio is better than at $\mathsf{hw}(x) = s$. Furthermore, the solution is more likely to be unique. Therefore, we are in fact interested in inputs of Hamming weight *up to* $s$, for some $s$.

**Property 3** (Recursive structure)**.** We can repeat the process of querying interesting inputs and exhaustive partial-key search on different input-output pairs, until we recover the entire key. Notice that after recovering $u$ key elements out of $n$, the remaining task reduces to a smaller instance of the same key recovery problem, where the function's key size is now $n - u$ instead of $n$.

In the weak-PRF setting, there is of course a tradeoff between the rarity of low-weight inputs and the cost of the exhaustive search ; we explore the optimal choice of $s$ in Section 3.3.

### 3.2 Querying cost

We introduce some shorthand notation for describing the key and the attacker's knowledge thereof.

- $K$ denotes the attacker's partial knowledge of the key $k$:

$$K_i = \begin{cases} k_i & \text{if the attacker has recovered } k_i \\ \perp & \text{otherwise} \end{cases}$$

– $\bar{K}$ denotes the bitmask of the unknown key elements:

$$\bar{K}_i = \begin{cases} 0 & \text{if the attacker has recovered } k_i \\ 1 & \text{otherwise} \end{cases}$$

– $\mathsf{nz}(x)$ denotes the set of indices $i$ such that $x_i \neq 0$.
– $u^{(i)}$ denotes the number of known key elements at the start of the $i$-th round.
– $\bar{u}^{(i)} = n - u^{(i)}$ denotes the number of unknown key elements at the start of the $i$-th round.

For notational convenience, we write $u$ in place of $u^{(i)}$ when the dependency on the round index $i$ is clear from context or not essential to the analysis. Similar rule applies to $\bar{u}^{(i)}$ so we have $\bar{u}$.

In the wPRF setting, the attacker is not in control of the function inputs ; the latter are instead provided by a random oracle.

*Initialization.* At attack start $\bar{u} = n$. The probability to obtain a Hamming-weight $s$-out-of-$n$ or less vector from the uniform random oracle is

$$\frac{\sum_{i=0}^{s} \binom{n}{i}(p-1)^i}{p^n}.$$

*Partial-key recovery.* At the start of the $i$-th round of the partial-key search, if $u$ out of $n$ key elements have already been recovered, we seek wPRF inputs $x$ such that $\mathsf{hw}(x \odot \bar{K}) \leq s$ for some $s$.

The probability to get a weight $s$ or less vector among $\bar{u}$ unknown elements is now

$$\frac{\sum_{i=0}^{s} \binom{\bar{u}}{i}(p-1)^i}{p^{\bar{u}}},$$

i.e. we reduce to a smaller problem instance where $n$ becomes $\bar{u} < n$, as per Property 3. The expected number of queries to obtain $\mathsf{hw}(x) \leq s$, denoted as $C_q(s)$, is thus the inverse of this probability:

$$C_q(s) = p^{\bar{u}} \cdot \left[\sum_{i=0}^{s} \binom{\bar{u}}{i}(p-1)^i\right]^{-1}. \tag{1}$$

### 3.3 Optimizing $s$ for low cost

For simplicity, we consider that an oracle query has the same cost as a local wPRF computation. To fine-tune the attack, one might weigh those differently, but the attack principle is unchanged.

With $1 \leq \bar{u} \leq n$ unknown elements, the expected total cost $C(s)$ as a function of the choice of $s$ is

$$C(s) = C_q(s) + C_e(s) = p^{\bar{u}} \cdot \left[\sum_{i=0}^{s} \binom{\bar{u}}{i}(p-1)^i\right]^{-1} + p^s, \tag{2}$$

where the exhaustive search cost on $s$ elements is $C_e(s) = p^s$.

Some facts about $C(s)$:

- $C_e(s)$ grows exponentially with $s$, from $C_e(0) = 1$ to $C_e(n) = p^n$.
- $C_q(s)$ decreases from $C_q(0) = p^n$ to $C_q(n) = 1$. In-between, $C_q(s)$ decreases in a convex manner.
- $C_q(s)$ and $C_e(s)$ are both convex, thus $C(s)$ is also convex.

In Fig. 1, we depict the trends of $C_q$ and $C_e$ for $n = 256$ and $p = 2, 3$.

For a given number of unknown key elements $\bar{u}$, the unique optimal value $s^*$ for $s$ is

$$s^* = \arg \min_{1 \leq s \leq \bar{u}} C(s). \tag{3}$$

Thus, the complete attack goes as follows: while some key elements are unknown, we find the integer minimum $s^*$ of the convex function $C(s)$ with the current $\bar{u}$, and then query the wPRF until we receive an input whose Hamming weight over unknown positions is at most $s^*$.

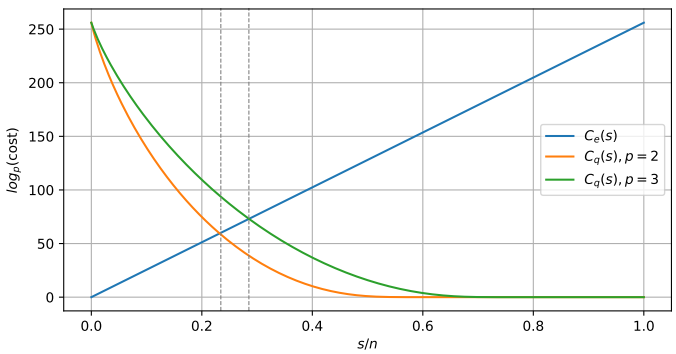


Fig. 1: Trends of $C_q$ and $C_e$ for $n = 256$ and $p = 2, 3$. Highlighted are the optimal values $s^*$ for each prime. Because of the exponential scale, $C_e + C_q$ is minimized practically at the intersection point.

Experimentally, by naively minimizing Eq. (3) for various $n$, we notice that $s^*$ is roughly proportional to $n$, i.e. $s^* \approx n\sigma_p$ for some constant $\sigma_p$ that depends on $p$ (and only lightly on $n$, while in the low-$n$ regime). In the following, we study the behavior of $s^*$ via the ratio $\sigma = s/n$ and its optimum $\sigma^*$.

### 3.4 Behavior of $s^*$

The ratio $s^*/n$ is a crucial parameter to determine the complexity of our attack.

Now that the attack strategy has been laid out, we study a single attack round from the perspective of the start of the attack, when $\bar{u} = n$. Subsequent

rounds are simply a smaller instance, with a lower $\bar{u}$, of the same problem, per Property 3.

**Bounding $\mathsf{Vol}_p$.** In the query cost $C_q(s) = p^{\bar{u}} \cdot [\sum_{i=0}^{s} \binom{\bar{u}}{i}(p-1)^i]^{-1}$, we recognize the summation as the volume $\mathsf{Vol}_p(n, s)$ of a *Hamming ball* of radius $s$ in the vector space $\mathbb{F}_p^n$ [1]:

$$\mathsf{Vol}_p(\bar{u}, s) = \sum_{i=0}^{s} \binom{\bar{u}}{i}(p-1)^i$$

Therefore the total cost can be rewritten as:

$$C(s) = C_q(s) + C_e(s) = p^{\bar{u}} \cdot \mathsf{Vol}_p^{-1}(\bar{u}, s) + p^s.$$

We use the following coding theory result to analytically bound $\mathsf{Vol}_p(n, s)$:

**Proposition 1 ([MS77]).** *For $0 \leq \delta \leq 1 - \frac{1}{p}$, for large enough $n$, we have*

$$\frac{1}{n+1} p^{nH_p(\delta)} \leq \mathsf{Vol}_p(n, n\delta) \leq p^{nH_p(\delta)}, \tag{4}$$

*where $H_p(x) = x \log_p(p-1) - x \log_p x - (1-x) \log_p(1-x)$ is the $p$-ary entropy function.*

From Proposition 1, we get the approximation of the total cost:

$$C(s) \approx p^s + (n+1)p^{n(1-H_p(s/n))}.$$

**Equal exponents method.** We may argue that due to the respective exponential growth trends of $C_e$ and $C_q$ with $s$, the arg min of their sum is not very sensitive to relatively small approximations. Equating the terms $C_e$ and $C_q$, $\sigma_p$ is such that

$$\begin{aligned} p^{n\sigma_p} &= p^n \mathsf{Vol}_p^{-1}(n, n\sigma_p) \\ &= p^n \cdot p^{-nH_p(\sigma_p)} \end{aligned}$$

Thus,

$$\begin{aligned} n\sigma_p &= n - nH_p(\sigma_p) \\ \sigma_p &= 1 - H_p(\sigma_p) \end{aligned} \tag{5}$$

This simple method is missing yields an implicit formula for $\sigma_p$, but is missing the low-$n$ regime. The following method is more precise in the low-$n$ regime but yields the same asymptotic result.

[1] The zero input vector is not interesting, as its output does not yield any key information. However, the associated exhaustive search cost is zero, so there is no harm in keeping it in consideration at this stage.

**Differential method.** Our objective is to minimize the total cost $C(s)$, where $s = n\sigma$. We derive an implicit expression of an upper bound for the minimizing $\sigma^*$ (using the lower bound in Eq. (4)). The function $C(n\sigma)$ attains its minimum when the first derivative satisfies $\frac{\mathrm{d}}{\mathrm{d}\sigma}C(n\sigma^*) = 0$.

$$\begin{aligned} C'(n\sigma) &= n \ln p \cdot p^{n\sigma} - n \ln p \cdot (n+1) \cdot p^{n(1-H_p(\sigma))} \cdot H'_p(\sigma) \\ &\propto p^{n\sigma} - (n+1) \cdot p^{n(1-H_p(\sigma))} \cdot H'_p(\sigma) \end{aligned}$$

with $H'_p(x) = \frac{\mathrm{d}}{\mathrm{d}x} H_p(x) = \log_p(p-1) + \log_p \frac{1-x}{x}$.
Setting to zero:

$$0 = p^{n\sigma^*} - (n+1) \cdot p^{n(1-H_p(\sigma^*))} \cdot H'_p(\sigma^*)$$

We have $\sigma^* < \frac{p-1}{p}$ so $H'_p(\sigma^*) > 0$:

$$\begin{aligned} 0 &= p^{n\sigma^*} - (n+1) \cdot p^{n(1-H_p(\sigma^*))+\log_p(n+1)+\log_p H'_p(\sigma^*)} \\ 0 &= \sigma^* - (1 - H_p(\sigma^*)) - \frac{\log_p(n+1) + \log_p H'_p(\sigma^*)}{n} \end{aligned} \tag{6}$$

Similarly, an implicit lower bound can be obtained by taking the upper bound in Eq. (4):

$$0 = \sigma^* - (1 - H_p(\sigma^*)) - \frac{\log_p H'_p(\sigma^*)}{n} \tag{7}$$

In Eq. (6), the last term $\frac{\log_p(n+1)+\log_p H'_p(\sigma^*)}{n}$ goes to zero as $n$ grows, converging to the formula yielded by the equal exponents method. The upper and lower bounds close in on each other, and the remaining equation has a single solution on $\left[0, \frac{p-1}{p}\right]$ by the intermediate value theorem. This confirms that $\sigma^*$ converges to some constant $\sigma_p$ that depends only on $p$.

**Theoretical Summary.** For large $n$, the ratio $\sigma^* = s^*/n$ only depends on $p$; we call this value $\sigma_p$ defined by $\sigma_p = 1 - H_p(\sigma_p)$. Its value starts at $\sigma_2 \approx 0.227, \sigma_3 \approx 0.281$ and very gently grows toward 0.5 (*e.g.* $\sigma_p \approx 0.49$ for $p \approx 2^{15}$).

**Experiments for $\boldsymbol{\sigma_p}$.** In practice, $s^*$ is an integer, and $s^*/n$ is quantized; however, we observe that $s^*/n$ indeed tends to $\sigma_p$. An experimental visualization of $s^*$ and $\sigma_p$ is provided in Fig. 2.

An attacker needs to compute $s^*$ – if naively using Eq. (3) is not doable because $n$ is too large or computation with large integer is unavailable, one may reduce the search space to a handful of integers between the upper and lower bound provided by implicit equations Eq. (6) and Eq. (7).

### 3.5 Total cost

Recall that $C(s)$ depends not on $n$ but on the number of unknown key elements $\bar{u}$. Therefore, every time we decrease $\bar{u}$ by revealing key elements, the remainder of the problem reduces to a smaller instance.

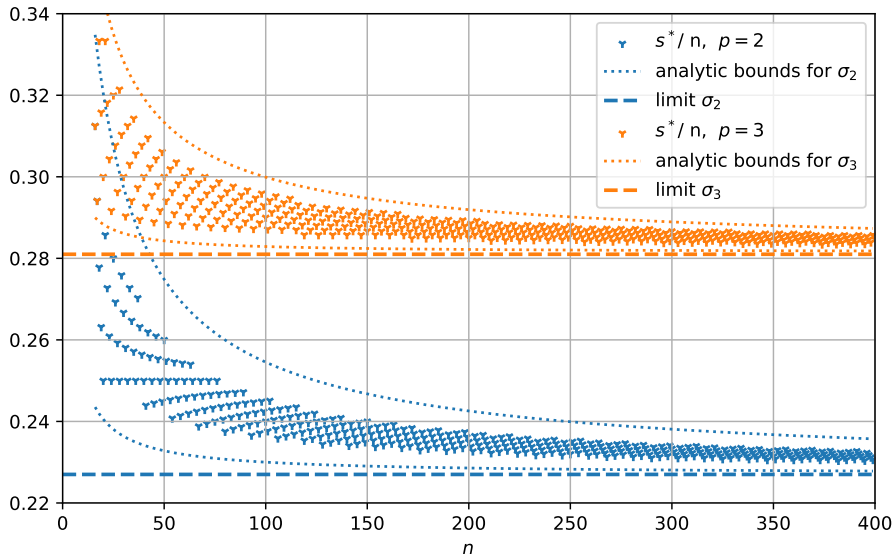


Fig. 2: Experimental $s^*/n$ (naturally quantized), between the analytic bounds for $\sigma^*$ (per Eq. (6) (7)), and above their limiting value $\sigma_p$, as a function of $n$, for $p = 2$ and $p = 3$.

For the complexity analysis, we set the optimal $s$ value minimizing total cost to $s^* = \sigma_p \cdot \bar{u}$. In the low-$n$ regime, the true $s^*$ may be slightly greater.

We may assume that we find inputs with exactly the threshold $s^*$ useful key elements (i.e. $\mathsf{hw}(x \odot \bar{K}) = s^*$) every time – this is a (slightly) pessimistic assumption because of Property 2. Then, the evolution of $\bar{u}$ after $i$ successful rounds is a simple geometric progression:

$$\begin{aligned} \bar{u}^{(0)} &= n \\ \bar{u}^{(i+1)} &= \bar{u}^{(i)} - s^{*(i)} \approx (1-\sigma_p)\bar{u}^{(i)} \\ \Rightarrow \bar{u}^{(i)} &\approx n \cdot (1-\sigma_p)^i \end{aligned}$$

and for $i \geq 0$, $s^{*(i)} = n\sigma_p(1-\sigma_p)^i$.

Thanks to the choice of $s^*$, we have $C_e(s^*) \approx C_q(s^*)$. The attack ends when $\bar{u} \leq 1$ so after approximately $r = -\ln n / \ln(1-\sigma_p) = \log_{\frac{1}{1-\sigma_p}}(n)$ successes.

So the total cost can be estimated as

$$\mathcal{C} = 2\sum_{i=0}^{r-1} C_q(s^{*(i)}) \approx 2\sum_{i=0}^{r-1} C_e(s^{*(i)}) \approx 2\sum_{i=0}^{r-1} p^{n\sigma_p(1-\sigma_p)^i}.$$

The log-complexity of each round decreases exponentially. The super-exponential decay of the cost at each round means that the dominating cost is the first round ; this is confirmed experimentally.

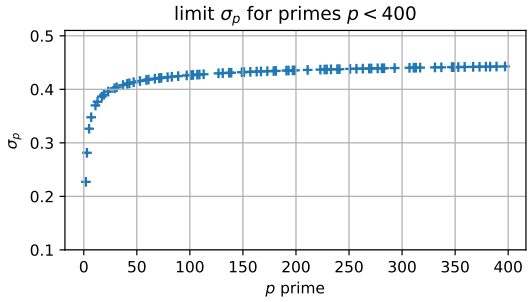


Fig. 3: Trend of $\sigma_p$ for large $n$, slowly growing to $1/2$.

We now try to derive a simpler bound. To get rid of the inner exponent $(1-\sigma_p)^i$, we draw the chord between the sum bounds $i = 0$ and $i = r-1$, and substitute the inner exponent with its chord value (i.e. linear interpolation) between 1 and $(1-\sigma_p)^{r-1}$, which yields an upper bound.

$$\begin{aligned}
\mathcal{C} &\approx 2\sum_{i=0}^{r-1} p^{n\sigma_p(1-\sigma_p)^i} \\
&< 2\sum_{i=0}^{r-1} \left(p^{n\sigma_p}\right)^{1+\frac{(1-\sigma_p)^{r-1}-1}{r-1}\cdot i} \quad \text{(linearize the exponent)} \\
&= 2p^{n\sigma_p}\sum_{i=0}^{r-1}\left[p^{n\sigma_p\frac{(1-\sigma_p)^{r-1}-1}{r-1}}\right]^i, \quad \text{let } a(n) \triangleq p^{n\cdot\sigma_p\cdot\frac{(1-\sigma_p)^{r-1}-1}{r-1}} \\
&= 2p^{n\sigma_p}\sum_{i=0}^{r-1} a(n)^i = 2p^{n\cdot\sigma_p}\frac{1-a(n)^r}{1-a(n)} \le 2p^{n\cdot\sigma_p}\cdot 2, \quad \text{if } a(n) < 1/2.
\end{aligned} \tag{8}$$

Now rewrite $a(n)$ by substituting the expected $r = -\log_{1-\sigma_p}(n)$:

$$a(n) = p^{n\sigma_p\cdot\frac{1-(1-\sigma_p)^{r-1}}{1-r}} = p^{n\sigma_p\cdot\frac{1-\frac{1}{n(1-\sigma_p)}}{1+\ln(n)/\ln(1-\sigma_p)}} = p^{\sigma_p\cdot\frac{n-\frac{1}{1-\sigma_p}}{1+\ln(n)/\ln(1-\sigma_p)}}.$$

The expression for $a(n)$ is quite messy, so we simply show it shrinks fast enough. The exponent is $\sim -\frac{n}{\ln n}$, so $a(n)$ is decreasing. With hypothetical worst-case parameters $p = 2$, $\sigma_p = 0.2$, then for $n > 100$ we get $a(n) < 1/2$.[2]

This derivation shows that despite requiring multiple rounds, and because of the double exponent, the cost of the attack is dominated by that of the first round.

[2] Any other threshold can be substituted, yielding a different constant factor and minimum $n$, but leaving the asymptotic behavior unchanged.

Finally, the total complexity of our attack is

$$\mathcal{C} < 4 \cdot p^{n \cdot \sigma_p} \ \sim \ \mathcal{O}(p^{n \cdot \sigma_p}).$$

### 3.6 Full algorithms

During the partial-key exhaustive search, it is possible that multiple partial-key candidates $\{\hat{k}^{(i)}\}_i$ are such that $F(\hat{k}^{(i)}, x) = y$. Note that by construction, $x$ is the first input we encounter that is zero everywhere but on $\mathsf{nz}(x)$ ; so we cannot use previously encountered input-output pairs to disambiguate the candidates.

When a round concludes with several candidate partial-keys, we call it a *failure*, as we cannot recover any key elements for certain. Conversely, a *success* occurs when the exhaustive partial-key search yields a single partial-key (i.e. the correct one).

We propose two algorithms for two distinct cases:

- **Case 1:** the wPRF is such that failures occur rarely enough that they do not prohibitively delay the attack. Against such wPRFs, we may use the simpler Algorithm 1, where we simply ignore failures.
- **Case 2:** the wPRF is such that failures occur very frequently, and it is desirable to attempt to leverage them. Against such wPRFs, we should use the more complex Algorithm 2, which introduces an additional *cross-check* step, but can eventually turn failures into successes.

We conclude by evaluating the conditions for one case or the other.

#### 3.6.1 Case 1: Unique partial-key solution

If the wPRF's image space is large enough compared to the input size $p^n$, there will typically be a single partial key candidate at each round, i.e. failures occur with negligible probability.

This is the scenario assumed by the complexity analysis in Section 3.5.

In this scenario, we simply ignore failures and carry on. The attack algorithm is then simple and is detailed in Algorithm 1.

#### 3.6.2 Case 2: Multiple partial-key solutions and disambiguation

If the wPRF's image space is too small, many rounds will be wasted in failures. It is however possible to disambiguate failures, provided some additional logic.

Consider two candidate partial keys $\hat{k}$ and $\hat{k}'$ from different rounds, i.e. they lie respectively over masks $w$ and $w' \in \{0,1\}^n$. We say they *disagree*, or that they are *incompatible*, if $\hat{k} \odot w' \neq \hat{k}' \odot w$. Else, they *agree* or are *compatible*: they bear no contradiction with each other.

If two candidate partial keys are incompatible, they cannot both be correct.

We take the following approach:

**Algorithm 1:** Pointwise-keyed wPRF attack, without cross-checking

**Input:** oracle $\mathcal{O}$; wPRF family $F$; parameters $n, t$
**Output:** the wPRF key $K$

```
ū ← n                                   // number of unknown key elements
K ← (⊥, ..., ⊥)                         // key accumulator
K̄ ← (1, ..., 1)                         // unknown element mask
while ū > 0 do
    s* ← Get-Optimal-s(ū, p)            // obtain from Eq. (3)
    repeat
        x, y ← O
        w ← nz(x) ⊙ K̄                   // active mask
        s ← hw(w)
    until s ≤ s*
    S ← {}                              // set of candidate partial-keys
    for k̂ ← each of the p^s possible sub-keys do
        if F_k̂(x) = y then
            S.add(k̂)
    if |S| = 1 then
        ū ← ū − s                       // update key knowledge
        for i ∈ nz(w) do
            K_i ← k̂_i
            K̄_i ← 0
return K
```

1. When a failure occurs, we save the list $\{\hat{k}_i\}_{i\in[N]}$ of $N$ candidate partial-keys and the *active mask* $w$ of the positions they concern.
2. At the end of each round, we get a list $S$ of $N'$ candidate partial-keys $\{k'_j\}_{j\in[N']}$ (be it of length 1 for successes). Recall that exactly one of the candidate partial-keys is correct in each list of candidates, therefore if a candidate partial-key disagrees with the entirety of another list, it cannot be correct. We cross-check the candidates lists of previous failures with the new list: each candidate $\hat{k} \in S$ must agree with at least one candidate $\hat{k}'$ in each other list $S'$, and conversely. In other words, $\hat{k}$ can be crossed out if there exists a common active index $j \in w \odot w'$ such that $\hat{k}_i \neq \hat{k}'_j$, or equivalently, if $\hat{k} \odot w' \neq \hat{k}' \odot w$. This corresponds to Line 18 to Line 24 in Algorithm 2.
3. Once a candidate list is reduced to a single candidate, we interpret it as a success and recover the corresponding key elements. More generally, when all partial keys in a list agree at some position $i$, then $k_i$ is no longer ambiguous and can be recovered early. This corresponds to Line 25 to Line 32 in Algorithm 2.

This *failure cross-check* allows to eventually extract key elements from failures. Because the cross-check is mutual, it can result in two (or more) successive failures resolving each other.

**Algorithm 2:** Pointwise-keyed wPRF attack, with cross-checking

**Input:** oracle $\mathcal{O}$; wPRF family $F$; parameters $n, t$
**Output:** the wPRF key $K$

```
ū ← n                                          // number of unknown key elements
K ← (⊥, ..., ⊥)                                // key accumulator
K̄ ← (1, ..., 1)                                // unknown element mask
L ← {}                                         // database of unresolved candidate lists
while ū > 0 do
    s* ← Get-Optimal-s(ū, p)                   // obtain from Eq. (3)
    repeat
        x, y ← O
        w ← nz(x) ⊙ K̄                          // active mask
        s ← hw(w)
    until s ≤ s*
    S ← {}                                     // set of candidate partial-keys
    for k̂ ← each of the p^s possible sub-keys do
        if F_k̂(x) = y then
            S.add(k̂)
    L.add((S, w))                              // save S and its active mask
    repeat                                     // start cross-checking
        for (S', w') ∈ L \ {(S, w)} do
            if w' ⊙ w = (0, ..., 0) then
                Continue
            // check the new list S against the past list S'
            // linear-time if done by table lookup on the bits w ⊙ w'
            for k̂ ∈ S do
                If k̂ disagrees with every k̂' ∈ S', remove it
            for k̂' ∈ S' do
                If k̂' disagrees with every k̂ ∈ S, remove it
        // check whether any S now has constant bit positions
        for (S, w) ∈ L do
            if S was modified in the previous loop then
                while ∃ i ∈ w', ∃ b ∈ F_p s.t. ∀k̂ ∈ S : k̂_i = b do
                    ū ← ū − 1                  // we learn k_i = b
                    K_i ← b
                    K̄_i ← 0
                    w_i ← 0                    // remove bit i from the active mask of S
                Shrink S correspondingly to the new w
            if S is empty (w = 0) then
                remove (S, w) from L           // prune resolved lists
    until all lists stay unchanged
return K
```

The cross-check can be done in time linear in the list sizes (i.e. time $N + N'$) by organizing one list into a hash map keyed by the common elements $\hat{k} \odot w \odot w'$, then querying the same common positions from the other list.

At the end of each round, we check the outcome of each new round against each candidate list awaiting resolution from prior failed rounds.

Algorithm 2 describes the augmented attack. The parts identical to Algorithm 1 are grayed out to highlight the new parts.

**Failure probability.** Given input $x \in \mathbb{F}_p^n$ of Hamming weight $s$, we can consider the pointwise-keyed wPRF $F$ applied on input $x$ as a function $g_x(k)$ of the key over the $s$ *active* input elements (now from $\mathbb{F}_p^s$ to $\mathbb{F}_q^t$).

Assuming $g$ is itself pseudorandom, then for any $k' \neq k \in \mathbb{F}_p^s$, the probability $\Pr[g_x(k) = g_x(k')]$ is $\frac{1}{q^t}$. Then for a fixed $k$, there are $p^s - 1$ other possible partial keys, and the number $N$ of other candidate partial-keys that also map $x$ to $y$ follows a Poisson distribution with parameter $\mu_1 = \frac{p^s-1}{q^t} \approx p^s q^{-t}$. Thus the expected total candidate list size is $\mu = 1 + \mu_1 \approx 1 + p^s q^{-t}$.

The probability to get multiple candidate partial-keys for an input-output pair $(x, y)$ obtained from the oracle is thus

$$\Pr[\text{failure}] \triangleq 1 - e^{-\mu_1} = 1 - e^{-\frac{p^s-1}{q^t}}.$$

If $\Pr[\text{failure}]$ is small (i.e. $\mu_1 > 1$), then Algorithm 1 is suitable ; we are in Case 1. Otherwise, Algorithm 1 is simply slowed by a factor $\Pr[\text{failure}]$, as it ignores failures ; and Algorithm 2 may bring valuable speedup.

**Cross-check complexity.** We saw above that the expected number of partial key candidates found during the exhaustive search over $s$-out-of-$n$ input elements is $\mu = p^s q^{-t}$. During the cross-check process where $s \leq n\sigma_p$, we need to store and iterate over lists of this size.

The complexity of cross-check in a single round is approximately $\mathcal{O}(|L| \cdot \mu)$, where $|L|$ considers the expected maximum number of failures that linger in the database $L$ (Line 4) before being resolved (Line 34). However, this is quite nontrivial to estimate in terms of $\Pr[\text{failure}]$ [3]. We conjecture it is less than $(1 - \Pr[\text{failure}])^{-1}$.

At the very least, for the cross-check to be tractable, $p^{n\sigma_p} q^{-t}$ must be sufficiently small in both time and memory.

## 4 Application to concrete weak PRF constructions

### 4.1 Attacks against weak PRFs in [APRR24]

The alternating-moduli wPRFs defined in [APRR24] are pointwise-keyed functions ; in this subsection, we attack them via the partial key method. The parameters they proposed were previously reported in Table 1.

[3] Doing so would likely require considering the statistics of the intersection of random subsets, over multiple rounds, between every pair of lists. The specific statistics of each candidate list also depend on the wPRF itself.

**One-to-One parameter set.** The Standard ($p = 2, q = 3$) and Reversed ($p = 3, q = 2$) variants respectively specify $n = 2\lambda$ and $n = \frac{2\lambda}{\log_2 3}$.

With $\sigma_2 = 0.227$ and $\sigma_3 = 0.281$, the respective complexities are thus $\mathcal{O}(2^{0.45\lambda})$ and $\mathcal{O}(2^{0.56\lambda})$.

At $t = \frac{2\lambda}{\log_2 3}$ and $t = 2\lambda$, the expected candidate list size $\mu \approx p^{n\sigma_p} q^{-t}$ is much less than 1 in both variants. This is confirmed experimentally: no failure is observed, so the cross-check process is not necessary, and Algorithm 1 suffices.

We show the concrete complexity of each round for an execution of the attack on the One-to-One Standard parameters with $\lambda = 28$, in Fig. 4 (for this parameter set, the failure probability is negligible and each round successfully yields key elements).

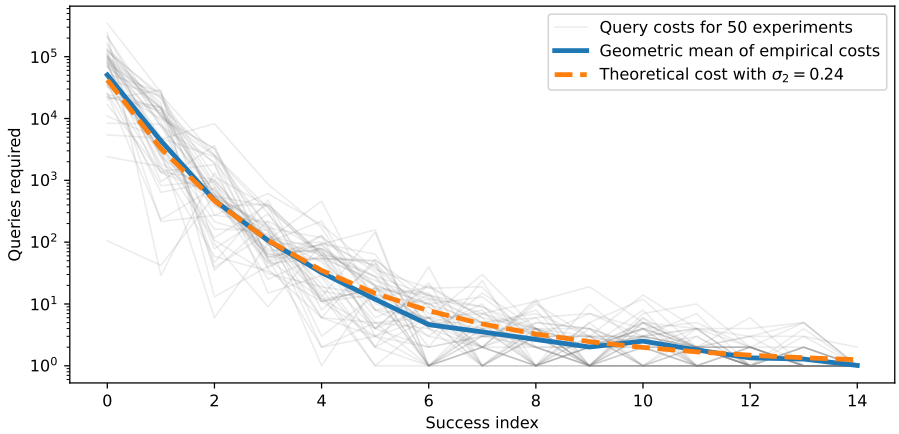


Fig. 4: Log of number of queries, One-to-One parameters Standard variants, $\lambda = 28$ ($n = 56$). Note the $y$-axis is logarithmic. In this regime of low $n$, $\sigma_p = 0.24$ is more accurate than the limiting value we give elsewhere for large $n$. Observe the first success dominates the attack cost.

**Many-to-One parameter set, Standard variant.** The Standard variant specifies $n = 4\lambda$. With the same $\sigma_2 = 0.227$, we get a complexity $\mathcal{O}(2^{0.91\lambda})$.

With $t = \frac{\lambda}{\log_2 3}$, the expected candidate list size $\mu \approx p^{n\sigma_p} q^{-t} = 2^{(0.91-1)\lambda} = 2^{-0.09\lambda}$ is bounded: we don't expect to see many failures, nor Algorithm 2 to be useful. This is also confirmed experimentally.

**Many-to-One parameter set, Reversed variant.** Here, $n = \frac{4\lambda}{\log_2 3}$. With $\sigma_3 = 0.281$, the query and search complexity using partial-key search is $2^{1.12\lambda}$, exceeding $2^{\lambda}$ ; the attack fails.

For the sake of completeness, we also study the effectiveness of the cross-check in this scenario. With $t = \lambda$, the expected candidate list size $\mu \approx p^{n\sigma_p} q^{-t} =$

$2^{(1.12-1)\lambda} = 2^{0.12\lambda}$ grows with $\lambda$. However, it stays manageable (for example $\lambda = 128$, $\mu \approx 2^{15.9}$). So here, Algorithm 2 is very useful, in that without it, the running time would suffer a large penalty factor $(1 - \Pr[\text{failure}])^{-1}$ when $\Pr[\text{failure}] = 1 - e^{-\mu_1}$ is close to 1.

We summarize our attacks on wPRFs, and the complexities obtained in Section 4.1.

| Variants | One-to-One | | Many-to-One | |
|---|---|---|---|---|
| | Zeroed-out | Partial-key[‡] | Zeroed-out | Partial-key[‡] |
| Standard $(\mathbb{F}_2, \mathbb{F}_3)$-wPRF | $\mathcal{O}(2^{0.5\lambda} \log_2 \lambda)$ | $\mathcal{O}(2^{0.45\lambda})$ | ✗ | $\mathcal{O}(2^{0.91\lambda})$ |
| Reversed $(\mathbb{F}_3, \mathbb{F}_2)$-wPRF | $\mathcal{O}(2^{0.84\lambda} \log_2 \lambda)$ | $\mathcal{O}(2^{0.56\lambda})$ | ✗ | ✗ |

[‡] $\sigma_p = 0.227$ and 0.281 for the Standard and Reversed parameters, shown in Fig. 2.

### 4.2 Effectiveness against fixes to Zeroed-out

Zeroed-out exploited the reduction in output space caused by the 0-values in the key, which induces sub-mappings that are far from the intended One-to-One mappings. To ensure that no part of the input is zeroed out as the first operation of the wPRF, they suggested excluding 0's as coefficients in the key; i.e. sampling them in $\mathbb{F}_p^*$, where prime $p > 2$.

Applying our attack to this scenario, the query cost $C_q(s)$ does not change, but the partial-key exhaustive search complexity on $s$ bits decreases from $C_e(s) = p^s$ to $C_e^{\mathsf{fix}}(s) = (p-1)^s$; and the expected cost becomes:

$$C^{\mathsf{fix}}(s) = C_q(s) + C_e^{\mathsf{fix}}(s) < C(s).$$

The new minimum of $C^{\mathsf{fix}}(s)$ now occurs at a larger $s^*$ value (i.e. $\sigma_p^{\mathsf{fix}} > \sigma_p$), still at the point where $C_q(s) \approx C_e^{\mathsf{fix}}(s)$. The total cost becomes $\mathcal{C} \approx 2C_e^{\mathsf{fix}}(s) = 2(p-1)^{n\sigma_p^{\mathsf{fix}}}$. In any case, $C^{\mathsf{fix}}(s) < C(s)$ and thus $\mathcal{C}^{\mathsf{fix}} < \mathcal{C}$. Experimentally, we observe that as $p$ grows, $\sigma_p^{\mathsf{fix}}$ tends toward $\sigma_p$. In summary, our attack is at least as powerful on the nonzero-key countermeasure as without, assuming identical parameters, and the difference shrinks as $p$ grows and $(p-1)/p$ goes to 1.

### 4.3 Applicability to the alternative wPRF in [BIP+18]

Boneh *et al.* [BIP+18] proposed an alternative weak PRF which is suitable for the two-party distributed evaluation. The secret key is $k \in \mathbb{F}_2^n$, and for an input $x \in \mathbb{F}_2^n$, the output is defined as:

$$F(k, x) := \sum_{i \in [n]} k_i x_i \bmod 2 + \sum_{i \in [n]} k_i x_i \bmod 3 \pmod 2 \in \mathbb{F}_2.$$

In [CCKK21], it was suggested to increase the size of input $n$ from 384 to 610 or 1220 for 128-bit security to prevent their attacks.

This wPRF is pointwise-keyed with $f : w \mapsto w \oplus (w \bmod 3) \bmod 2$, but in contrast to the wPRFs attacked above, the output is binary ($t = 1$, $p = q = 2$). In turn, the probability of failure becomes $\approx 1 - e^{-2^{n-1}}$, i.e. close to 1.

Applying our method to this wPRF, the expected candidate list size is $\mu \approx p^{n\sigma_p} q^{-t} = 2^{n\sigma_2 - 1} \approx 2^{0.227n}$. In order to run within reasonable memory space, *e.g.* $< 2^{48}$, the higher $n$ that can feasibly be attacked is $n = 211$.

However, for small $n < 40$, we experimentally observe that the cross-check process yields its first key elements after as many rounds as there are candidate partial-keys, i.e. after $\approx \mu$ rounds. If this trend generalizes to larger $n$, the time complexity of the cross-check process approaches $\mu^2 \approx p^{2n\sigma_p - 2} \approx p^{0.45n}$, which exceeds $2^\lambda$ for the proposed parameters.

## 5 Final remarks

### 5.1 Comparison to Zeroed-out

Applying our attack on the class of wPRFs vulnerable to Zeroed-out, i.e., both the Standard and the Reversed variants in One-to-One parameter sets, we get the following improvements:

- We have improved asymptotic time and data complexity on the wPRFs attacked by Zeroed-out.
- On the One-to-One parameter set, our attack requires data complexity proportional to $p^{n\sigma_p} = 2^{2\lambda\sigma_p}$ ($p = 2, 3$). Zeroed-out has data complexity $\sqrt{2^{h_1}} \log \lambda$ where $h_1$ is the Hamming weight of the key ; according to $p$, this translates into $2^{\lambda \frac{p-1}{p}} \log \lambda$, which is larger for $p = 2$ and $p = 3$ (although the gap would tighten if $p$ were larger).
- Our attack is memoryless when $p^{n\sigma_p} < q^t$ ; in contrast with the collision table that Zeroed-out needs to store.
- Our memory complexity when $p^{n\sigma_p} > q^t$ resides in the storage of candidate lists and is proportional to $p^{n\sigma_p} q^{-t} \cdot |L|$ where $L$ is the number of rounds required before the first key elements are recovered ; this is decreasing in $t$. On the other hand, the collision table in Zeroed-out has roughly size $\sqrt{2|\operatorname{Im}(F)| \cdot c} \approx q^{t/2}\sqrt{2c}$ with $c$ is the required number of collisions, which grows with $t$. The memory requirement of our attack and of Zeroed-out have inverse sensitivity to the wPRF output size.

For the Many-to-One Standard variants, the input space size $2^{4\lambda}$ significantly exceeds the output space size $2^\lambda$, so collisions are inevitable. However, collisions occur with significantly higher probability in $f = \mathbf{B}_{\cdot q} \circ \mathbf{A}_{\cdot p}$ than in pointwise multiplication $k \odot_p x$. It will take $\mathcal{O}(2^\lambda)$ time for Zeroed-out to generate even a single collision in $k \odot_p x$. Our attack is more general: it can still target pointwise-keyed wPRFs that exhibit collisions downstream of the pointwise product, or when $q^t < p^{n\sigma_p}$, which Zeroed-out struggles with.

### 5.2 Restoring the security against partial-key search

We discuss potential ways to restore the security of the wPRFs against the partial-key attacks in Section 4.

**Forgo the pointwise operation.** The main weakness exploited in our attacks and Zeroed-out, is the pointwise multiplication ($k \odot_p x$) operation that occurs before the alternating moduli maps. To increase the strength of these wPRFs, Manterola Ayala and Raddum [MR25] propose to either replace this operation with addition $\bmod p^n$, or another operation such that no input bit becomes irrelevant. The countermeasures are effective in addressing the vulnerabilities revealed through our attacks.

**Larger $p$.** We saw that increasing $p$ gently nudges $\sigma_p$ toward 0.5 (Fig. 3), so increasing $p$ improves security against our attack. For example, if $n = \text{poly}(\lambda)/\log_2(p)$, as in the wPRF parameters proposed in [APRR24], the size of the input set stays roughly constant, but security against our attack increases thanks to the increase in $\sigma_p$. This is what put the Reversed Many-to-One variant out of our attack's reach. Increasing $q$ does not improve security at all, on the contrary.

## 6 Conclusion

In this paper, we proposed a cryptanalysis of the alternating moduli wPRF proposed in [APRR24] that improves on the recent Zeroed-out attack : we give improved attack complexity, in both time and memory, and we successfully attack one of the two Many-to-One parameter sets. The method can be applied to other wPRFs that follow the same input-key pointwise product paradigm.

Functions where the input-to-output compression ratio is high make it difficult to distinguish the correct partial key among a large number of candidates. We provided a cross-check process that alleviates the limitation of our method at high compression ratios and extends its practical range. Functions such as the alternative wPRF (Section 4.3), with high compression ratio, remain out of reach of our attack. Conversely, functions where the compression ratio is low are especially vulnerable, with $\mathcal{O}(1)$ memory requirement and no slowdown due to failures.

This attack and the Zeroed-out attack show that the pointwise operation between key and input tends to be a significant concern. On the other hand, in neither attack is the alternating-moduli paradigm at fault ; this paradigm is still at this time a promising design strategy to balance efficiency and security. Finally, the Many-to-One Reversed parameter set still resists our attack. A cryptanalysis of it could shed further light on the resilience of alternating moduli constructions in different configurations.

**Acknowledgments.** We thank the anonymous reviewers of ASIACRYPT 2025 for their insightful comments and suggestions, which helped to improve the presentation of this paper. We also thank Yixin Shen and Weiqiang Wen for the discussion related to this work.

## References


ABG+14. Adi Akavia, Andrej Bogdanov, Siyao Guo, Akshay Kamath, and Alon Rosen. Candidate weak pseudorandom functions in $AC^0 \circ MOD_2$. In Moni Naor, editor, *Innovations in Theoretical Computer Science, ITCS'14,*, pages 251–260. ACM, 2014.

ADDG24. Martin R. Albrecht, Alex Davidson, Amit Deo, and Daniel Gardham. Crypto dark matter on the torus - oblivious prfs from shallow prfs and TFHE. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 447–476. Springer, 2024.

App14. Benny Applebaum. Bootstrapping obfuscators via fast pseudorandom functions. In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology - ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 162–172. Springer, 2014.

APRR24. Navid Alamati, Guru-Vamsi Policharla, Srinivasan Raghuraman, and Peter Rindal. Improved alternating-moduli PRFs and post-quantum signatures. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024, Part VIII*, volume 14927 of *LNCS*, pages 274–308. Springer, 2024.

BCK96. Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Keying hash functions for message authentication. In Neal Koblitz, editor, *Advances in Cryptology - CRYPTO '96*, volume 1109 of *LNCS*, pages 1–15. Springer, 1996.

BCM+24. Dung Bui, Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Fast public-key silent OT and more from constrained naor-reingold. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 88–118. Springer, 2024.

Bel06. Mihir Bellare. New proofs for NMAC and HMAC: security without collision-resistance. In Cynthia Dwork, editor, *Advances in Cryptology - CRYPTO 2006*, volume 4117 of *LNCS*, pages 602–619. Springer, 2006.

BIP+18. Dan Boneh, Yuval Ishai, Alain Passelègue, Amit Sahai, and David J. Wu. Exploring crypto dark matter: - new simple PRF candidates and their applications. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part II*, volume 11240 of *LNCS*, pages 699–729. Springer, 2018.

CCKK21. Jung Hee Cheon, Wonhee Cho, Jeong Han Kim, and Jiseung Kim. Adventures in crypto dark matter: Attacks and fixes for weak pseudorandom functions. In Juan A. Garay, editor, *Public-Key Cryptography - PKC 2021, Part II*, volume 12711 of *LNCS*, pages 739–760. Springer, 2021.

CDD+24. Geoffroy Couteau, Lalita Devadas, Srinivas Devadas, Alexander Koch, and Sacha Servan-Schreiber. QuietOT: lightweight oblivious transfer with a public-key setup. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024, Part II*, volume 15485 of *LNCS*, pages 197–231. Springer, 2024.

DGH+21. Itai Dinur, Steven Goldfeder, Tzipora Halevi, Yuval Ishai, Mahimna Kelkar, Vivek Sharma, and Greg Zaverucha. MPC-friendly symmetric cryptography from alternating moduli: Candidates, protocols, and applications. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 517–547. Springer, 2021.

DMMS21. Sébastien Duval, Pierrick Méaux, Charles Momin, and François-Xavier Standaert. Exploring crypto-physical dark matter and learning with physical rounding towards secure and efficient fresh re-keying. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(1):373–401, 2021.

DP10. Yevgeniy Dodis and Krzysztof Pietrzak. Leakage-resilient pseudorandom functions and side-channel attacks on feistel networks. In Tal Rabin, editor, *Advances in Cryptology - CRYPTO 2010*, volume 6223 of *LNCS*, pages 21–40. Springer, 2010.

GGM86. Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *J. ACM*, 33(4):792–807, 1986.

Gol86. Oded Goldreich. Two remarks concerning the Goldwasser-Micali-Rivest signature scheme. In Andrew M. Odlyzko, editor, *Advances in Cryptology - CRYPTO '86*, volume 263 of *LNCS*, pages 104–110. Springer, 1986.

HMM+23. Clément Hoffmann, Pierrick Méaux, Charles Momin, Yann Rotella, François-Xavier Standaert, and Balazs Udvarhelyi. Learning with physical rounding for linear and quadratic leakage functions. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 410–439. Springer, 2023.

JMN23. Thomas Johansson, Willi Meier, and Vu Nguyen. Differential cryptanalysis of mod-2/mod-3 constructions of binary weak PRFs. In *IEEE International Symposium on Information Theory, ISIT 2023*, pages 477–482. IEEE, 2023.

MR25. Irati Manterola Ayala and Håvard Raddum. Zeroed out: Cryptanalysis of weak prfs in alternating moduli. *IACR Trans. Symmetric Cryptol.*, 2025(2):1–15, 2025.

MS77. F. J. MacWilliams and N. J. A. Sloane. *The Theory of Error-Correcting Codes*, volume 16 of *North-Holland Mathematical Library*. North-Holland, 1977.

# A Detailed Complexity Analysis of the Cross-check Process

We study the complexity of the cross-check process in the regime of high failure probability ($p^{n\sigma_p}q^{-t}$ large or increasing in $\lambda$). The time until the first key bit is recovered is a measure of choice, as the remaining key recovery becomes much faster. The estimation gets involved, and in the end we use Monte Carlo simulation to estimate the result.

We aim to answer the question: how many failures are necessary before the cross-check yields its first key bit?

Reminder: a candidate list $S$ contains 1 correct key and a number $\sim \text{Poisson}(\mu_1)$ wrong keys. For a general analysis, we assume wrong keys to be uniformly random. In this section, we refer to the optimum $\sigma^*$ as simply $\sigma$.

*Random subset intersection.* Let $A, B$ independent and uniformly random subsets of sizes $a, b$ of a $n$-element set. Then $|A \cap B|$ follows a hypergeometric distribution with parameters $(n, a, b)$ (or equivalently $(n, b, a)$). Its expected value is $\mathbb{E}\left[|A \cap B|\right] = \frac{ab}{n}$.

Thus, during cross-check, two candidate lists over two subsets of $n\sigma$ bits have on average $n\sigma^2$ common bits.

*List reduction.* Consider a wrong key $\hat{k} \in S'$ being cross-checked with a candidate list $S'$. $S, S'$ have sizes respectively $|S|, |S'|$ and have $h$ bit positions in common.

$\hat{k}$ will be compared to each subkey $\hat{k}' \in S'$ only on the common bit positions. How many distinct strings of common bits are there in $S'$? This is a balls-into-bins problems: there are $|S'|$ balls being thrown into $p^h$ bins. In expectation there are $l' = p^h\big(1 - (1 - p^{-h})^{|S'|}\big) \approx p^h\big(1 - e^{-|S'|/p^h}\big)$ occupied bins.

*Wrong key detection.* A wrong key $\hat{k} \in S$ gets eliminated if no $\hat{k}' \in S'$ agrees on the common positions. Thus the probability that $\hat{k}$ gets eliminated is $f = (1 - p^{-h})^{l'} \approx e^{-l'/p^h}$. This process applies almost[4] independently to each wrong key in $S$. The number of remaining wrong keys would follow a binomial distribution with parameters $(|S|, 1 - f)$ and expected value $|S|(1 - f)$.

*Multiple candidate lists.* Given $\phi + 1$ failures, each list gets cross-checked with $\phi$ other lists. The cross-checking process applies to list $S$ a number of $\phi$ times, once per other list, each of which contains a different number of candidates drawn from Poisson($\mu_1$). To simplify things, we set them all to size $\mu_1$ [5]. Then the expected number of remaining wrong keys in $S$ is $N_{\text{rem}} = |S|(1 - f)^{\phi}$.

*Effectiveness of constant-bit detection.* When a list gets small, it is likely that some bit positions become constant and some bits can be recovered early (second for loop of Algorithm 2). How small a list is required? If a list $S$ is reduced to $c$ candidates, each bit position has probability $p \cdot p^{-c}$ to hold the same value in all $c$ candidates. Among the $\sigma n$ active positions of $S$, the probability that at least one of them is constant is $1 - (1 - p^{-c+1})^{n\sigma} \approx 1 - e^{-n\sigma/p^{c-1}}$. Thus the maximum list size that keeps $\Omega(1)$ probability to find a constant bit position is $c \approx 1 + \log_p(n\sigma)$.

*Probability that some list gets small enough.* After cross-checking with $\phi$ other lists, each given list $S$ is expected to contain $N_{\text{rem}}$ remaining wrong keys. This number is small enough with probability $\rho = \text{Poisson}(\mu_1).\text{cdf}(N_{\text{rem}})$ (denoting the Poisson cumulative distribution function). To have $\Omega(1)$ probability that at least one list gets small enough, we need $\phi = \Omega(1/\rho)$.

We estimate $\phi$ using Monte Carlo simulation on the Reversed Many-to-One parameters ($n = 4\lambda/\log_2(3)$, $t = \lambda$, $p = 3$, $q = 2$). Recall that this parameter set is different from the three that were successfully attacked, as here $\mu_1 = p^{n\sigma}q^{-t}$ is increasing in $\lambda$. We give the estimated values for instance with small $\lambda$, and the corresponding $\mu_1$ in Table 2.

[4] in the sense that wrong candidate keys are considered uniformly random, but distinct, and thus not statistically independent.

[5] We argue that this is a conservative approximation, as trading two average-length lists for one smaller- and one larger-than-average list is beneficial to reach a short list earlier.

Table 2: Averages from Monte-Carlo simulation of some of the above random variables for the Reversed Many-to-One parameters ($10^5$ samples).

| $\lambda$ | 8 | 16 | 20 | 24 | 28 | 32 | 36 | 40 | 44 | 48 | 64 | 96 | 128 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\lvert S'\rvert \approx \mu_1$ | 2.8 | 8.1 | 13.7 | 23.1 | 39.0 | 21.9 | 37.0 | 62.4 | 105.3 | 177.7 | 480.5 | $1.1 \cdot 10^4$ | $2.3 \cdot 10^5$ |
| $h$ | 1.8 | 3.6 | 4.5 | 5.4 | 6.3 | 6.6 | 7.5 | 8.4 | 9.2 | 10.1 | 13.1 | 19.7 | 26.23 |
| $N_{rem}$ | 1.1 | 1.9 | 2.7 | 3.8 | 5.5 | 1.9 | 2.8 | 4.0 | 6.0 | 8.7 | 9.2 | 43.5 | 209.67 |
| $\phi$ | 1.2 | 1.3 | 1.4 | 1.5 | 1.6 | 1.2 | 1.3 | 1.3 | 1.3 | 1.3 | 1.2 | 1.2 | 1.0 |

The expected values for $\phi$ stay very low, i.e. only $1 + \phi \approx 2$ or 3 failures are required to get the first key bit(s). That is, the complexity of the outer `repeat` loop in Algorithm 2 is very low for these parameters.

These numbers match our experiments quite well in a tractable range ($\lambda \leq 20$), but we could not verify them experimentally for larger values.

Our code can be in `MonteCarlo.py` at:

https://gitlab.telecom-paris.fr/c2/pointwise-keyed-wprf-attack

In conclusion, Algorithm 2 is effective at palliating the shortcomings of Algorithm 1 when the parameters are such that $\mu_1 = p^{n\sigma}q^{-t}$ grows slow enough compared to $p^h \approx p^{n\sigma^2}$. It is our best attempt at an extension of our attack to the Reversed Many-to-One parameter set, and it successfully eliminates the failure rate overhead for a very modest computational complexity (candidate lists have manageable sizes).

We expect that for other, unproposed parameter sets, where $\mu_1$ is increasing but $t$ grows slower, the cross-check process will require many more failures to recover any bit. These can be simulated using the same Python code.